

Labs 8-9: In-Class Task

Django & Django REST Framework

Duration: 40 minutes

Task: Build a Movie Database API

Create a Django REST API for managing movies and directors. The API should support full CRUD operations using Django REST Framework.

Requirements

Part 1: Project Setup (5 minutes)

1. Create a new Django project named `movie-api`
 2. Create a virtual environment and activate it
 3. Install Django and djangorestframework
 4. Create an app named `movies`
 5. Register the app and `rest_framework` in `INSTALLED_APPS`
-

Part 2: Define Models (10 minutes)

Create the following models in `movies/models.py`:

Director model:

- `first_name` — CharField (max 50 characters)
- `last_name` — CharField (max 50 characters)
- `birth_year` — IntegerField
- `country` — CharField (max 50 characters)

Movie model:

- `title` — CharField (max 200 characters)
- `release_year` — IntegerField
- `genre` — CharField (max 50 characters)
- `rating` — FloatField (1.0 to 10.0)
- `duration_mins` — IntegerField

- `director` — ForeignKey to Director (on_delete=CASCADE)

Add `__str__` method to both models.

Run migrations after creating models.

Part 3: Create Serializers (5 minutes)

Create `movies/serializers.py` with:

DirectorSerializer:

- Use ModelSerializer
- Include all fields

MovieSerializer:

- Use ModelSerializer
 - Include all fields
 - Display director name (not just ID) in read operations
-

Part 4: Create ViewSets (10 minutes)

Create ViewSets in `movies/views.py`:

DirectorViewSet:

- Extend ModelViewSet
- Use DirectorSerializer
- Queryset includes all directors

MovieViewSet:

- Extend ModelViewSet
- Use MovieSerializer
- Queryset includes all movies

Custom Action:

- Add a custom action to DirectorViewSet
 - Endpoint: `/api/directors/<id>/movies/`
 - Returns all movies by a specific director
-

Part 5: Configure URLs (5 minutes)

Create `movies/urls.py`:

- Use `DefaultRouter`
- Register `directors` endpoint with `DirectorViewSet`
- Register `movies` endpoint with `MovieViewSet`

Include the app URLs in the project's `urls.py` with prefix `api/`.

Part 6: Admin & Test Data (5 minutes)

1. Register both models in `movies/admin.py`
 2. Create a superuser
 3. Add test data via admin panel:
 - At least 3 directors
 - At least 6 movies (2 per director)
-

API Endpoints

Your API should support these endpoints:

Method	Endpoint	Description
GET	/api/directors/	List all directors
POST	/api/directors/	Create a director
GET	/api/directors/{id}/	Get director by ID
PUT	/api/directors/{id}/	Update director
DELETE	/api/directors/{id}/	Delete director
GET	/api/directors/{id}/movies/	Get movies by director
GET	/api/movies/	List all movies
POST	/api/movies/	Create a movie
GET	/api/movies/{id}/	Get movie by ID
PUT	/api/movies/{id}/	Update movie
DELETE	/api/movies/{id}/	Delete movie

Expected JSON Responses

GET /api/directors/

```
json
[
  {
    "id": 1,
    "first_name": "Christopher",
    "last_name": "Nolan",
    "birth_year": 1970,
    "country": "UK"
  }
]
```

GET /api/movies/

```
json
```

```
[
  {
    "id": 1,
    "title": "Inception",
    "release_year": 2010,
    "genre": "Sci-Fi",
    "rating": 8.8,
    "duration_mins": 148,
    "director": 1
  }
]
```

GET /api/directors/1/movies/

json

```
[
  {
    "id": 1,
    "title": "Inception",
    "release_year": 2010,
    "genre": "Sci-Fi",
    "rating": 8.8,
    "duration_mins": 148,
    "director": 1
  },
  {
    "id": 2,
    "title": "Interstellar",
    "release_year": 2014,
    "genre": "Sci-Fi",
    "rating": 8.6,
    "duration_mins": 169,
    "director": 1
  }
]
```

File Structure

```
movie-api/
  manage.py
  movie_api/
    __init__.py
    settings.py
```

```
urls.py
wsgi.py
movies/
  __init__.py
  admin.py
  apps.py
  models.py
  serializers.py
  views.py
  urls.py
  migrations/
requirements.txt
.gitignore
```

Django/DRF Concepts Being Tested

- **Project & App creation** — django-admin startproject, python manage.py startapp
 - **Models** — CharField, IntegerField, FloatField, ForeignKey
 - **Migrations** — makemigrations, migrate
 - **ModelSerializer** — automatic serialization from models
 - **ModelViewSet** — full CRUD with minimal code
 - **DefaultRouter** — automatic URL routing
 - **Custom actions** — @action decorator for nested endpoints
 - **Admin registration** — admin.site.register
 - **ForeignKey relationships** — one-to-many (Director → Movies)
-

Grading Criteria

- Django project and app correctly created
- Both models defined with proper field types
- ForeignKey relationship working
- Migrations applied successfully
- Serializers created using ModelSerializer
- ViewSets created using ModelViewSet
- Custom action for director's movies working
- URLs configured with DefaultRouter
- All 11 endpoints functional

- Admin panel shows both models
- Test data added (3 directors, 6 movies)
- Code follows Django/Python conventions

Good luck!